Model-Based Reinforcement Learning

Learning Objectives

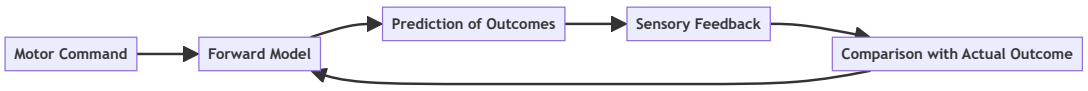
- Understand the role of models in reinforcement learning
- Learn how to construct and utilize forward models for prediction
- Explore Model-Predictive Control and its applications
- Implement Dyna-Q and Model-based Value Expansion techniques

Literature

This lecture draws from the following sources:

1. Eysenbach, COS435/ECE 433 Lecture 4: The Cross Entropy Method and Model-Predictive Control, 2025 (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-4-cem-mpc.pdf)
2. Eysenbach, COS435/ECE 433 Lecture 9: Learning Value Functions, 2025 (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-9-learning-value-functions.pdf)
3. Levine, CS285 Lecture 1: Deep Reinforcement Learning, Decision Making, and Control, 2023 (https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-1.pdf)
4. Levine, CS285 Lecture 10: Optimal Control and Planning, 2023 (https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-10.pdf)
5. Levine, CS285 Lecture 11: Model-Based Reinforcement Learning, 2018 (https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-11.pdf)
6. Sutton & Barto, Reinforcement Learning: An Introduction, 2nd Edition, 2018 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf)

Forward models and prediction in human cognition



Forward models in human cognition and psychology

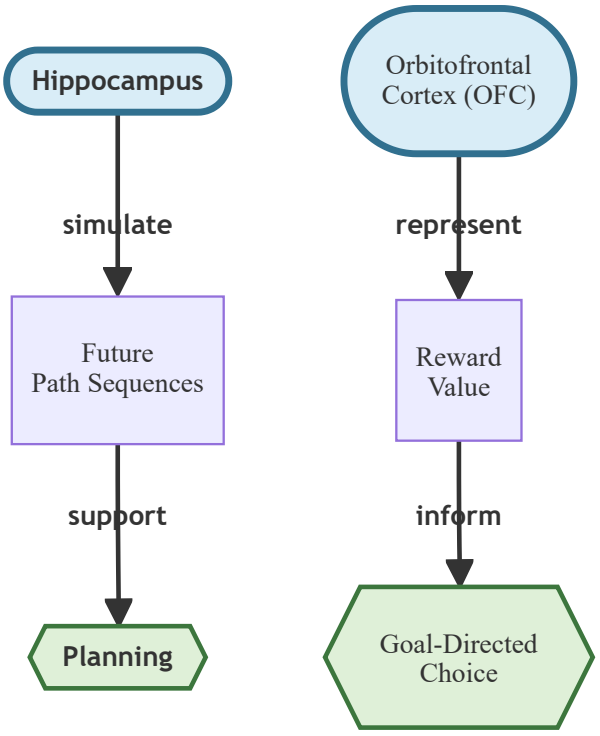
Prediction in sensorimotor control:
Forward models predict consequences of motor commands - central to human sensorimotor control. (Levine, CS285 Lecture 1, 2023, p. 49-66 (https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-1.pdf))

Cognitive maps:
Internal representations predicting state transitions and rewards - psychologists' "cognitive maps" align with model-based RL environment models. (Sutton & Barto, 2018, p. 385-390 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf))

Tolman's latent learning:
Rats develop cognitive maps of mazes without explicit rewards, using internal models to navigate efficiently once motivated. (Sutton & Barto, 2018, p. 385-390 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf))

Expectancy theory and system identification:
Psychology: Experiencing stimulus sequences builds expectations about future stimuli
Control engineering: Learning models from labeled training examples
Both concepts share the idea of prediction from experience. (Sutton & Barto, 2018, p. 385-390 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf))

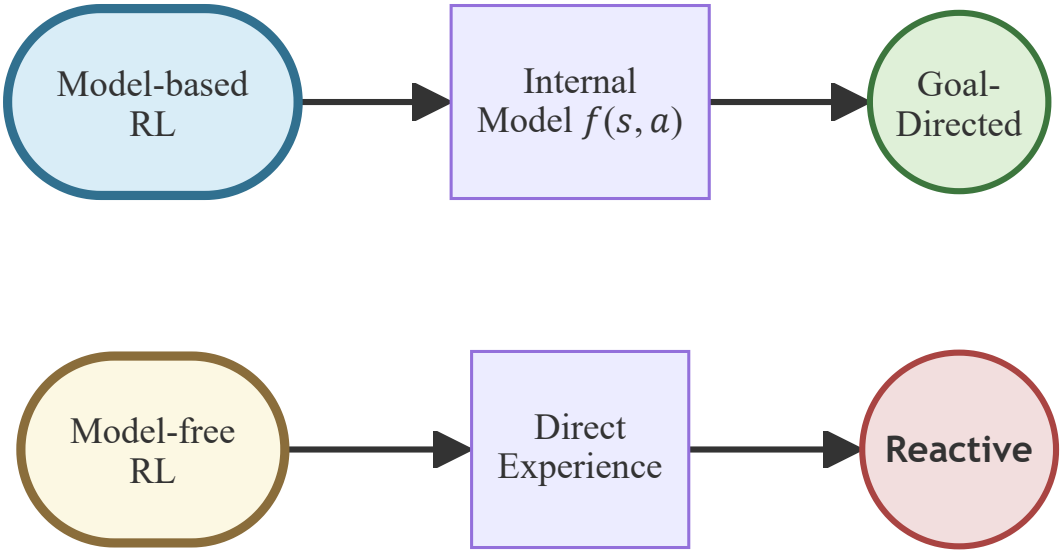
Forward models in neuroscience



Hippocampus and planning:
Decoding hippocampal activity in rats shows that at choice points, the hippocampus represents possible future paths, supporting planning through simulated state sequences. (Sutton & Barto, 2018, p. 429-431 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf))

Orbitofrontal cortex (OFC):
Represents subjective reward value of stimuli and expected rewards of actions, playing a key role in goal-directed choice. (Sutton & Barto, 2018, p. 429-431 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf))

Model-based vs. model-free RL



Model-based:
Uses internal models for proactive, goal-directed planning, enables rapid adaptation to environmental changes. (Sutton & Barto, 2018, p. 385-390 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf))

Model-free:
Relies on direct experience to update policies or value functions, corresponds to habitual behavior. (Sutton & Barto, 2018, p. 385-390 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf))

Note on terminology:
Sometimes the term "forward model" is used broadly to refer to any model of environment dynamics, including exact models used in Dynamic Programming methods (L4). However, in the context of **model-based RL**, we specifically refer to **learned forward models** - approximations of the true world dynamics learned from data, typically using function approximation (e.g., neural networks).

Learning forward models as supervised learning problem

Forward models as approximators for the world dynamics in an MDP

MDP definition:
A Markov Decision Process is defined by the tuple $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$ where:

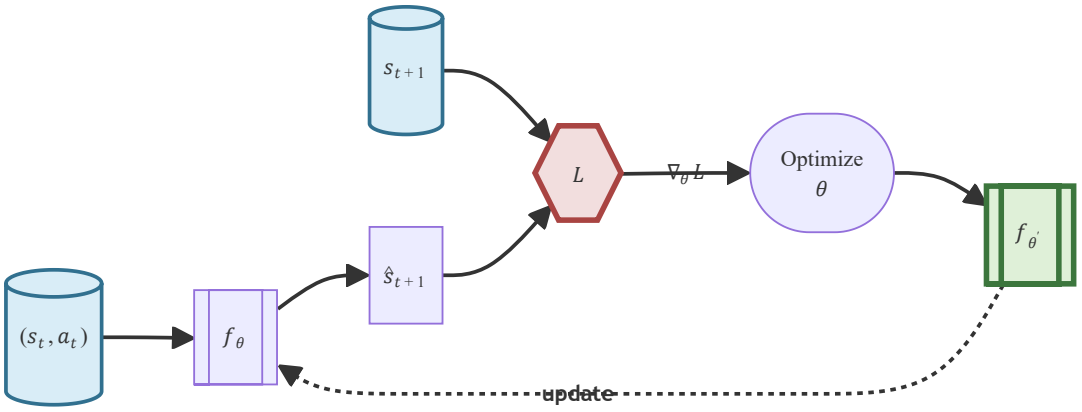
- $\mathcal{S}$ is the state space
- $\mathcal{A}$ is the action space
- $p(s'|s,a)$ is the transition dynamics (probability of next state s' given state s and action a)
- $r(s,a)$ is the reward function
- $\gamma \in [0,1]$ is the discount factor

Forward models approximate the dynamics:
The transition dynamics $p(s'|s,a)$ describes how the world evolves. In model-based RL, we learn a **forward model** f_ψ that approximates these dynamics:

- **Stochastic case**: $f_\psi(s'|s,a)$ approximates the full probability distribution $p(s'|s,a)$
- **Deterministic case**: $\hat{s}_{t+1} = f_\psi(s_t, a_t)$ approximates the expected next state

In this lecture, we will assume deterministic forward models if not stated otherwise.

Overview of supervised learning for forward models



Deterministic forward model:
A parameterized function $f_\psi : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$ (e.g., neural network) mapping state-action pairs to successor states:

$$\hat{s}_{t+1} = f_\psi(s_t, a_t)$$

where ψ are the model parameters (weights and biases).

Learning objective:
Minimize mean squared error between predicted and observed next states:

$$\min_\psi \mathcal{L}(\psi) = \min_\psi \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}} \left[\| f_\psi(s_t, a_t) - s_{t+1} \|^2 \right]$$

where $\mathcal{D}$ is the dataset of observed transitions collected by running a base policy (often exploratory). (Levine, CS285 Lecture 11, 2018, p. 1-18 (https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-11.pdf))

Online forward model learning algorithm

Algorithm: Online Forward Model Learning

Input: Policy π, environment Env, number of transitions N, learning rate α

Output: Learned forward model f_ψ

---

1. Initialize forward model f_ψ with random parameters ψ
2. Initialize state $s \leftarrow \mathrm{Env.reset}()$
3. **for** $t = 1$ **to** N **do**
4. Select action: $a \leftarrow \pi(s)$
5. Execute action in environment: $s' \leftarrow \mathrm{Env.step}(a)$
6. Predict next state: $\hat{s}' \leftarrow f_\psi(s, a)$
7. Compute loss: $\mathcal{L}(\psi) \leftarrow \|\hat{s}' - s'\|^2$
8. Update model parameters: $\psi \leftarrow \psi - \alpha \nabla_\psi \mathcal{L}(\psi)$
9. **if** episode terminated **then**
10. $s \leftarrow \mathrm{Env.reset}()$
11. **else**
12. $s \leftarrow s'$
13. **end if**
14. **end for**
15. **return** f_ψ

Key properties:

- **Online learning:** Model updated immediately after each transition
- **On-policy data:** Transitions generated by policy π during learning
- **Supervised learning:** MSE loss between predicted and observed next states
- **Continual adaptation:** Model improves as more data is collected

In [ ]:

```python
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import gymnasium as gym
import importlib

# Import utilities from helper files
import ddpg_utils
import helper_functions
importlib.reload(ddpg_utils)
importlib.reload(helper_functions)

from ddpg_utils import train_ddpg
from helper_functions import (
    plot_forward_model_errors,
    print_forward_model_summary,
    QNetwork,
    ReplayBuffer,
    select_action_epsilon_greedy
)


# ================================================================
# Forward Model (Dynamics Model)
# ================================================================

class ForwardModel(nn.Module):
    """Forward model f_ψ(s, a) → ŝ'"""

    def __init__(self, state_dim, action_dim, hidden_dim=128):
        super(ForwardModel, self).__init__()
        self.network = nn.Sequential(
            nn.Linear(state_dim + action_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, state_dim)
        )

    def forward(self, state, action):
        """Predict next state: ŝ' = f_ψ(s, a)"""
        x = torch.cat([state, action], dim=-1)
        return self.network(x)


# ================================================================
# Online Forward Model Learning
# ================================================================

def learn_forward_model_online(env, expert_actor, forward_model,
lr=1e-3,
```

```
                                num_transitions=5000, early_stop_
window=100):
    """
    Learn forward model online using expert policy to generate da
ta.

    Algorithm:
    1. Take forward model f_ψ as input
    2. For each timestep:
       a. Execute expert action: a = π_expert(s)
       b. Observe transition: (s, a, s')
       c. Predict next state: ŝ' = f_ψ(s, a)
       d. Compute loss: L = ||ŝ' - s'||²
       e. Update model: ψ ← ψ - α∇_ψ L
       f. Track error
    3. Early stopping if loss increases over window
    4. Return model and error history

    Args:
        env: Gym environment
        expert_actor: Trained DDPG actor (expert policy)
        forward_model: Forward model to train
        lr: Learning rate for forward model
        num_transitions: Number of transitions to collect
        early_stop_window: Window size for early stopping (defaul
t: 100)

    Returns:
        forward_model: Trained forward model
        errors: List of MSE errors over time
    """
    # Initialize optimizer for the forward model
    optimizer = optim.Adam(forward_model.parameters(), lr=lr)

    errors = []
    state, _ = env.reset()
    previous_done = False  # Track if previous step ended episode

    print(f"Learning forward model online (early stopping window:
{early_stop_window})...")

    for step in range(num_transitions):
        # Execute expert policy (without noise for deterministic
behavior)
        with torch.no_grad():
            state_tensor = torch.FloatTensor(state).unsqueeze(0)
            action = expert_actor(state_tensor).numpy()[0]

        # Observe transition
        next_state, reward, terminated, truncated, _ = env.step(a
ction)
        done = terminated or truncated

        # Convert to tensors for prediction (always compute for t
racking)
        state_tensor = torch.FloatTensor(state).unsqueeze(0)
        action_tensor = torch.FloatTensor(action).unsqueeze(0)
```

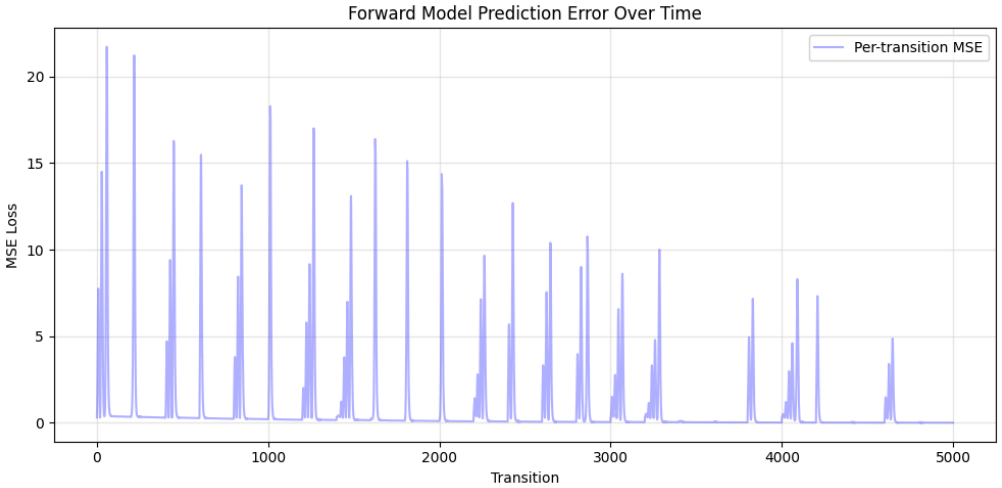
```
        next_state_tensor = torch.FloatTensor(next_state).unsquee
ze(0)

        # Predict next state
        predicted_next_state = forward_model(state_tensor, action
_tensor)

        # Compute MSE loss: L(ψ) = ||f_ψ(s,a) - s'||²
        loss = nn.MSELoss()(predicted_next_state, next_state_tens
or)

        # Only train on valid transitions (not following a reset)
        if not previous_done:
            # Update forward model
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

        # Track error for all transitions (to see where errors sp
ike)
        errors.append(loss.item())

        # Early stopping: check if loss is increasing over window
        if len(errors) >= 2 * early_stop_window:
            recent_mean = np.mean(errors[-early_stop_window:])
            previous_mean = np.mean(errors[-2*early_stop_window:-
early_stop_window])

            if recent_mean > previous_mean:
                print(f"\nEarly stopping at step {step + 1}/{num_
transitions}")
                print(f"  Previous mean loss: {previous_mean:.6
f}")
                print(f"  Recent mean loss: {recent_mean:.6f}")
                print(f"  Loss increased - stopping training")
                break

        # Progress reporting
        if (step + 1) % 500 == 0:
            avg_error = np.mean(errors[-500:]) if len(errors) >=
500 else np.mean(errors)
            print(f"Step {step + 1}/{num_transitions}, Avg MSE (l
ast 500): {avg_error:.6f}")

        # Update state for next iteration
        if done:
            state, _ = env.reset()
            previous_done = True
        else:
            state = next_state
            previous_done = False

    print("Forward model learning complete!")
    return forward_model, errors
```

In [54]:

```
# Setup environment
env = gym.make('Pendulum-v1')
state_dim = env.observation_space.shape[0]
action_dim = env.action_space.shape[0]

print(f"Environment: Pendulum-v1")
print(f"State dimension: {state_dim}")
print(f"Action dimension: {action_dim}")
print()

# Step 1: Train expert policy using DDPG
print("=" * 60)
print("STEP 1: Training Expert Policy (DDPG)")
print("=" * 60)
expert_actor = train_ddpg(
    env, state_dim, action_dim,
    num_episodes=200, max_steps=200,
    save_path="ddpg_model.pth",
    force_relearn_policy=False,  # Set to True to retrain from sc
ratch
    continue_training=False       # Set to True to load and conti
nue training
)
print("\nExpert policy training complete!\n")

# Step 2: Learn forward model online using expert policy
print("=" * 60)
print("STEP 2: Learning Forward Model Online")
print("=" * 60)

# Create forward model
forward_model = ForwardModel(state_dim, action_dim, hidden_dim=12
8)
print(f"Forward model created with hidden_dim=128")
print()

# Train the forward model
forward_model, errors = learn_forward_model_online(
    env, expert_actor, forward_model,
    lr=1e-5, num_transitions=5000, early_stop_window=2450
)

# Step 3: Plot forward model error over time
print("\n" + "=" * 60)
print("STEP 3: Visualizing Forward Model Learning")
print("=" * 60)

plot_forward_model_errors(errors, window_size=10000)
print_forward_model_summary(errors)

env.close()
```

```
Environment: Pendulum-v1
State dimension: 3
Action dimension: 1

============================================================
STEP 1: Training Expert Policy (DDPG)
============================================================
Loading existing policy from ddpg_model.pth (no additional traini
ng)

Expert policy training complete!

============================================================
STEP 2: Learning Forward Model Online
============================================================
Forward model created with hidden_dim=128

Learning forward model online (early stopping window: 2450)...
Step 500/5000, Avg MSE (last 500): 2.109865
Step 500/5000, Avg MSE (last 500): 2.109865
Step 1000/5000, Avg MSE (last 500): 1.029482
Step 1000/5000, Avg MSE (last 500): 1.029482
Step 1500/5000, Avg MSE (last 500): 1.617312
Step 1500/5000, Avg MSE (last 500): 1.617312
Step 2000/5000, Avg MSE (last 500): 0.771761
Step 2000/5000, Avg MSE (last 500): 0.771761
Step 2500/5000, Avg MSE (last 500): 1.118592
Step 2500/5000, Avg MSE (last 500): 1.118592
Step 3000/5000, Avg MSE (last 500): 0.915298
Step 3000/5000, Avg MSE (last 500): 0.915298
Step 3500/5000, Avg MSE (last 500): 0.795937
Step 3500/5000, Avg MSE (last 500): 0.795937
Step 4000/5000, Avg MSE (last 500): 0.254729
Step 4000/5000, Avg MSE (last 500): 0.254729
Step 4500/5000, Avg MSE (last 500): 0.502022
Step 4500/5000, Avg MSE (last 500): 0.502022
Step 5000/5000, Avg MSE (last 500): 0.196753
Forward model learning complete!

============================================================
STEP 3: Visualizing Forward Model Learning
============================================================

Forward Model Learning Summary:
  Initial MSE (first 100 steps): 4.457867
  Final MSE (last 100 steps): 0.003526
  Improvement: 99.92%
Step 5000/5000, Avg MSE (last 500): 0.196753
Forward model learning complete!

============================================================
STEP 3: Visualizing Forward Model Learning
============================================================

Forward Model Learning Summary:
  Initial MSE (first 100 steps): 4.457867
```

```
Final MSE (last 100 steps): 0.003526
Improvement: 99.92%
```



Accumulation of model errors

Small inaccuracies in each prediction step compound during multi-step planning, causing significant deviations from true dynamics. ([Levine, CS285 Lecture 11, 2018, p. 1-18 (https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-11.pdf)](https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-11.pdf))

Mathematical analysis of cumulative prediction error

Single-step prediction error:
Given true dynamics $p(s'|s,a)$ and forward model $f_\psi(s,a)$, the one-step prediction error is:

$$e_1 = \mathbb{E}_{s,a}\left[\|s' - \hat{s}'\|\right]$$

where $s' \sim p(\cdot|s,a)$ is the true next state and $\hat{s}' = f_\psi(s,a)$ is the predicted next state.

Recursive n-step prediction with policy:
Let s_t denote the true state and $\hat{s}_t$ the predicted state at time t. Given policy π and initial state s_0 :

$$\text{True trajectory:} \quad s_{t+1} \sim p(\cdot|s_t, \pi(s_t)) \tag{1}$$
$$\text{Predicted trajectory:} \quad \hat{s}_{t+1} = f_\psi(\hat{s}_t, \pi(\hat{s}_t)) \tag{2}$$

Recursive formula for n-step predicted state:

$$\hat{s}_0 = s_0 \quad \text{(initial state)} \tag{3}$$
$$\hat{s}_n = f_\psi(\hat{s}_{n-1}, \pi(\hat{s}_{n-1})) \quad \text{for } n \geq 1 \tag{4}$$

Cumulative error after n steps:

$$e_n = \|s_n - \hat{s}_n\|$$

Notation: We use e for prediction error to avoid confusion with ϵ (epsilon-greedy exploration parameter).

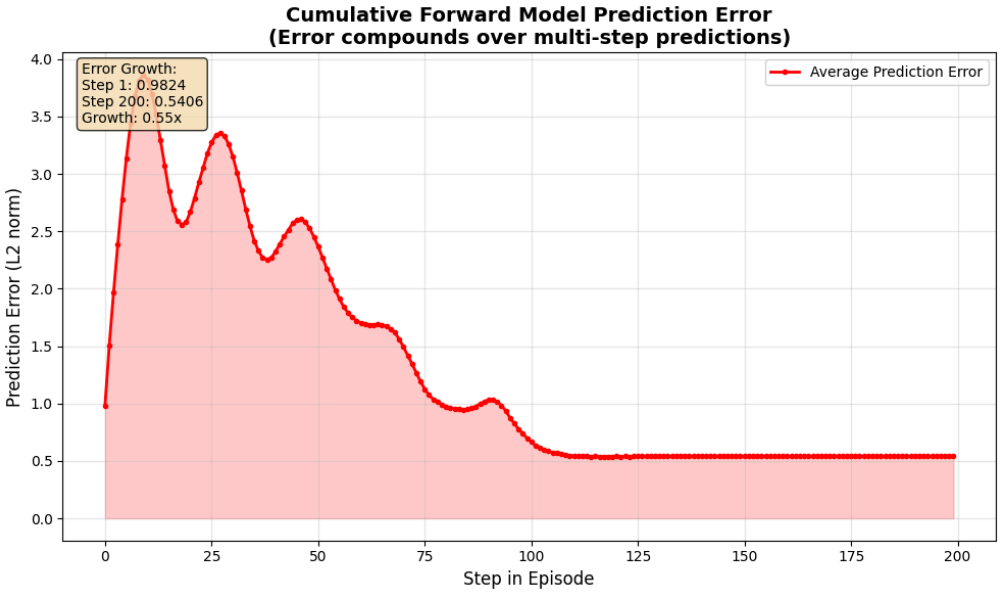
In [61]:
```
def evaluate_forward_model_cumulative_error(env, forward_model, p
olicy, num_episodes=50, max_steps=200):
    """
    Evaluate forward model by measuring cumulative prediction err
or over episodes.

    Compares predicted state trajectory (using forward model) vs
actual state trajectory
    (from environment) when following the same policy.

    Args:
        env: Gym environment
        forward_model: Trained forward model f_ψ(s,a) → ŝ'
        policy: Policy network π(s) → a
        num_episodes: Number of episodes to average over
        max_steps: Maximum steps per episode

    Returns:
        step_errors: Average error at each step (shape: max_step
s)
        step_counts: Number of episodes that reached each step
    """
    # Initialize error tracking
    error_accumulator = np.zeros(max_steps)
    step_counts = np.zeros(max_steps)

    print(f"Evaluating forward model cumulative error over {num_e
pisodes} episodes...")

    for episode in range(num_episodes):
        # Reset environment and get initial state
        s_real, _ = env.reset()
        s_pred = s_real.copy()  # Predicted state starts same as
real

        for t in range(max_steps):
            # Get action from policy using real state
            with torch.no_grad():
                state_tensor_real = torch.FloatTensor(s_real).uns
queeze(0)
                action_real = policy(state_tensor_real).numpy()
[0]

            # Get action from policy using predicted state
            with torch.no_grad():
                state_tensor_pred = torch.FloatTensor(s_pred).uns
queeze(0)
                action_pred = policy(state_tensor_pred).numpy()
[0]

            # Execute in real environment with action from real s
tate
            s_real_next, _, terminated, truncated, _ = env.step(a
ction_real)
            done = terminated or truncated
```

```
            # Predict next state using forward model with action
from predicted state
            with torch.no_grad():
                s_pred_tensor = torch.FloatTensor(s_pred).unsquee
ze(0)
                action_pred_tensor = torch.FloatTensor(action_pre
d).unsqueeze(0)
                s_pred_next = forward_model(s_pred_tensor, action
_pred_tensor).numpy()[0]

            # Compute prediction error at this step
            step_error = np.linalg.norm(s_real_next - s_pred_nex
t)
            error_accumulator[t] += step_error
            step_counts[t] += 1

            # Print current average error every 10 steps (first e
pisode only)
            if episode == 0 and t % 10 == 0:
                current_avg = error_accumulator[t] / step_counts
[t]
                print(f"  Step {t:3d}: Current avg error = {curre
nt_avg:.6f}")

            # Update states for next iteration
            s_real = s_real_next
            s_pred = s_pred_next

            if done:
                break

        if (episode + 1) % 10 == 0:
            print(f"  Episode {episode + 1}/{num_episodes} comple
te")

    # Compute average error per step
    step_errors = np.divide(error_accumulator, step_counts,
                            where=step_counts > 0, out=np.zeros_li
ke(error_accumulator))

    print("Forward model evaluation complete!")
    return step_errors, step_counts
```

In [62]:

```python
# Evaluate cumulative prediction error
print("=" * 60)
print("STEP 4: Evaluating Cumulative Prediction Error")
print("=" * 60)

# Reimport helper functions to get latest version
import importlib
importlib.reload(helper_functions)
from helper_functions import plot_cumulative_prediction_error

step_errors, step_counts = evaluate_forward_model_cumulative_erro
r(
    env, forward_model, expert_actor,
    num_episodes=50, max_steps=200
)

# Plot cumulative error growth
plot_cumulative_prediction_error(step_errors, step_counts)
```

```
============================================================
STEP 4: Evaluating Cumulative Prediction Error
============================================================
Evaluating forward model cumulative error over 50 episodes...
  Step   0: Current avg error = 0.925402
  Step  10: Current avg error = 5.295009
  Step  20: Current avg error = 1.274447
  Step  30: Current avg error = 6.560176
  Step  40: Current avg error = 1.747651
  Step  50: Current avg error = 0.501753
  Step  60: Current avg error = 0.192071
  Step  70: Current avg error = 0.279791
  Step  80: Current avg error = 0.262251
  Step  90: Current avg error = 0.260550
  Step 100: Current avg error = 0.260543
  Step 110: Current avg error = 0.260544
  Step 120: Current avg error = 0.260543
  Step 130: Current avg error = 0.260543
  Step 140: Current avg error = 0.260543
  Step 150: Current avg error = 0.260543
  Step 160: Current avg error = 0.260543
  Step 170: Current avg error = 0.260543
  Step 180: Current avg error = 0.260543
  Step 190: Current avg error = 0.260543
  Episode 10/50 complete
  Episode 10/50 complete
  Episode 20/50 complete
  Episode 20/50 complete
  Episode 30/50 complete
  Episode 30/50 complete
  Episode 40/50 complete
  Episode 40/50 complete
  Episode 50/50 complete
Forward model evaluation complete!
  Episode 50/50 complete
Forward model evaluation complete!
```



```
Out[62]: (<Figure size 1000x600 with 1 Axes>,
          <Axes: title={'center': 'Cumulative Forward Model Prediction Err
         or\n(Error compounds over multi-step predictions)'}, xlabel='Step
         in Episode', ylabel='Prediction Error (L2 norm)'>)
```

Techniques for improving model accuracy

Probabilistic methods: Ensembles and Bayesian approaches:
Average predictions across models or weight by confidence, reducing overfitting and improving robustness. (Levine, CS285 Lecture 11, 2018, p. 18-33 (https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-11.pdf))

--> We will discuss this in the next lecture L10

Predict state changes instead of states:
Rather than predicting $\hat{s}_{t+1} = f_\psi(s_t, a_t)$, predict the state change:

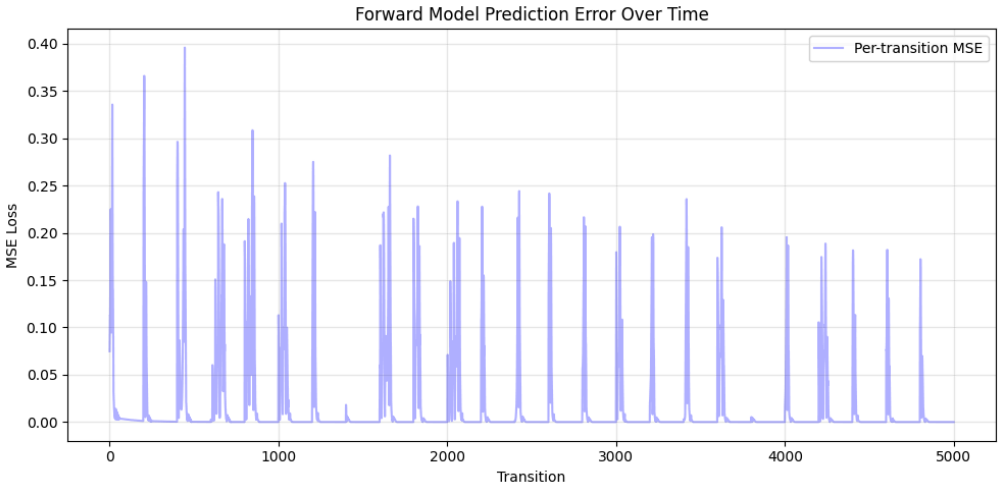
$$\Delta s = f_\psi(s_t, a_t), \quad \hat{s}_{t+1} = s_t + \Delta s$$

- **Zero-centered output:** State changes naturally centered around zero, easier to learn
- **Persistence assumption:** Aligns with physical intuition that most state components remain unchanged unless explicitly affected by actions
- **Reduced prediction magnitude:** Predicting small changes rather than full state values can improve numerical stability

In [ ]:

```
# ================================================================
=============
# Delta Forward Model (predicts state changes Δs)
# ================================================================
=============

class DeltaForwardModel(nn.Module):
    """Forward model that predicts state changes: Δs = f_ψ(s, a),
then returns s + Δs"""

    def __init__(self, state_dim, action_dim, hidden_dim=128):
        super(DeltaForwardModel, self).__init__()
        self.network = nn.Sequential(
            nn.Linear(state_dim + action_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, state_dim)
        )

    def forward(self, state, action):
        """Predict state change and add to current state: ŝ' = s
+ Δs"""
        x = torch.cat([state, action], dim=-1)
        delta_s = self.network(x)
        # Return s + Δs (no detach needed - gradients should flow
through delta_s only)
        return state + delta_s


# Train delta forward model
print("=" * 60)
print("Training Delta Forward Model")
print("=" * 60)

# Create delta forward model
delta_forward_model = DeltaForwardModel(state_dim, action_dim, hi
dden_dim=128)
print(f"Delta forward model created with hidden_dim=128")
print()

# Train the delta forward model
delta_forward_model, delta_errors = learn_forward_model_online(
    env, expert_actor, delta_forward_model,
    lr=1e-5, num_transitions=5000, early_stop_window=2450
)

# Plot delta forward model error over time
print("\n" + "=" * 60)
print("Visualizing Delta Forward Model Learning")
print("=" * 60)

plot_forward_model_errors(delta_errors, window_size=10000)
print_forward_model_summary(delta_errors)
```

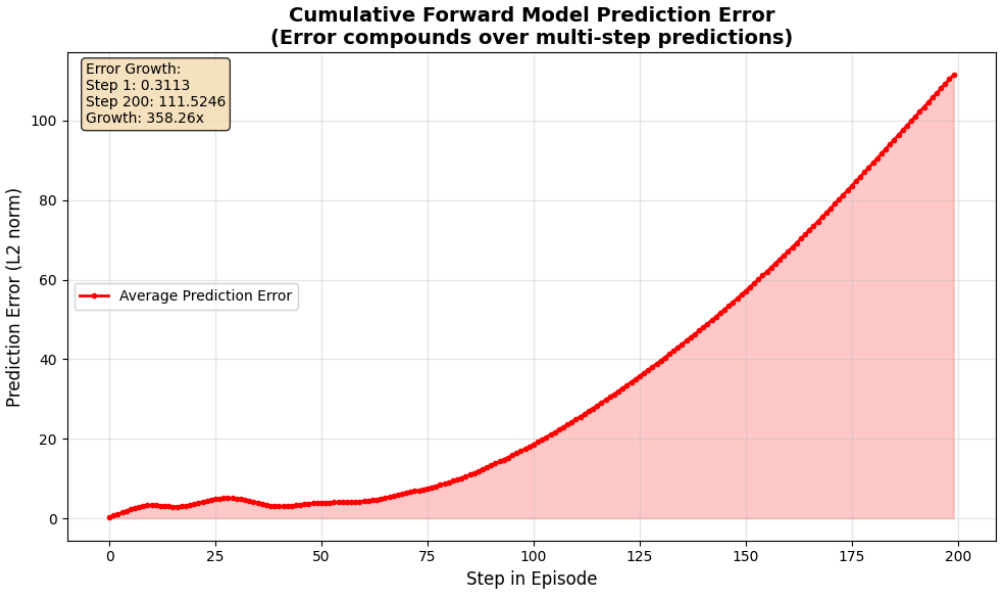
```
============================================================
Training Delta Forward Model
============================================================
Delta forward model created with hidden_dim=128

Learning forward model online (early stopping window: 2450)...
Step 500/5000, Avg MSE (last 500): 0.028418
Step 500/5000, Avg MSE (last 500): 0.028418
Step 1000/5000, Avg MSE (last 500): 0.028113
Step 1000/5000, Avg MSE (last 500): 0.028113
Step 1500/5000, Avg MSE (last 500): 0.016717
Step 1500/5000, Avg MSE (last 500): 0.016717
Step 2000/5000, Avg MSE (last 500): 0.022593
Step 2000/5000, Avg MSE (last 500): 0.022593
Step 2500/5000, Avg MSE (last 500): 0.022083
Step 2500/5000, Avg MSE (last 500): 0.022083
Step 3000/5000, Avg MSE (last 500): 0.010859
Step 3000/5000, Avg MSE (last 500): 0.010859
Step 3500/5000, Avg MSE (last 500): 0.017285
Step 3500/5000, Avg MSE (last 500): 0.017285
Step 4000/5000, Avg MSE (last 500): 0.007329
Step 4000/5000, Avg MSE (last 500): 0.007329
Step 4500/5000, Avg MSE (last 500): 0.015880
Step 4500/5000, Avg MSE (last 500): 0.015880
Step 5000/5000, Avg MSE (last 500): 0.006686
Forward model learning complete!

============================================================
Visualizing Delta Forward Model Learning
============================================================

Forward Model Learning Summary:
  Initial MSE (first 100 steps): 0.041279
  Final MSE (last 100 steps): 0.000005
  Improvement: 99.99%
Step 5000/5000, Avg MSE (last 500): 0.006686
Forward model learning complete!

============================================================
Visualizing Delta Forward Model Learning
============================================================

Forward Model Learning Summary:
  Initial MSE (first 100 steps): 0.041279
  Final MSE (last 100 steps): 0.000005
  Improvement: 99.99%
```


Forward Model Prediction Error Over Time
Per-transition MSE
MSE Loss
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0
1000
2000
3000
4000
5000
Transition

In [81]:

```
# Evaluate cumulative error for delta forward model
print("=" * 60)
print("Evaluating Delta Forward Model - Cumulative Error")
print("=" * 60)

delta_step_errors, delta_step_counts = evaluate_forward_model_cum
ulative_error(
    env, delta_forward_model, expert_actor,
    num_episodes=50, max_steps=200
)

# Plot cumulative prediction error for delta model
from helper_functions import plot_cumulative_prediction_error
plot_cumulative_prediction_error(delta_step_errors, delta_step_co
unts)

# Print errors at every 10 steps for analysis
print("\nError values at every 10 steps:")
for i in range(0, len(delta_step_errors), 10):
    if delta_step_counts[i] > 0:
        print(f"Step {i:3d}: {delta_step_errors[i]:.6f}")


print("This should be different! Any ideas?")
```

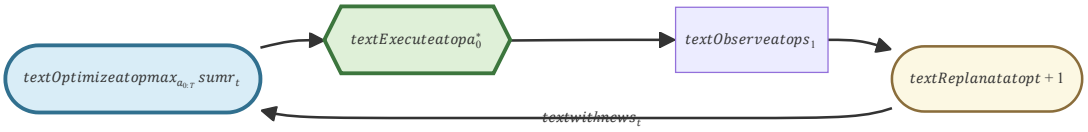
```
============================================================
Evaluating Delta Forward Model - Cumulative Error
============================================================
Evaluating forward model cumulative error over 50 episodes...
  Step   0: Current avg error = 0.346790
  Step  10: Current avg error = 6.869280
  Step  20: Current avg error = 1.676355
  Step  30: Current avg error = 2.540895
  Step  40: Current avg error = 2.710391
  Step  50: Current avg error = 2.963261
  Step  60: Current avg error = 4.136855
  Step  70: Current avg error = 7.979294
  Step  80: Current avg error = 16.113733
  Step  90: Current avg error = 27.374086
  Step 100: Current avg error = 39.978336
  Step 110: Current avg error = 53.240818
  Step 120: Current avg error = 66.908997
  Step 130: Current avg error = 80.834465
  Step 140: Current avg error = 94.935791
  Step 150: Current avg error = 109.176788
  Step 160: Current avg error = 123.536522
  Step 170: Current avg error = 138.017639
  Step 180: Current avg error = 152.613754
  Step 190: Current avg error = 167.335831
  Episode 10/50 complete
  Episode 10/50 complete
  Episode 20/50 complete
  Episode 20/50 complete
  Episode 30/50 complete
  Episode 30/50 complete
  Episode 40/50 complete
  Episode 40/50 complete
  Episode 50/50 complete
Forward model evaluation complete!
  Episode 50/50 complete
Forward model evaluation complete!
```



```
Error values at every 10 steps:
Step   0: 0.311298
Step  10: 3.298757
Step  20: 3.528269
Step  30: 4.912625
Step  40: 2.972913
Step  50: 3.909142
Step  60: 4.284852
Step  70: 6.327753
Step  80: 9.038543
Step  90: 13.284660
Step 100: 18.608447
Step 110: 24.880016
Step 120: 31.947588
Step 130: 39.670251
Step 140: 48.027959
Step 150: 57.161668
Step 160: 67.141983
Step 170: 77.935865
Step 180: 89.350779
Step 190: 101.078735
This should be different! Any ideas?
```

Model-Predictive Control

Overview of Model-Predictive Control



MPC as receding horizon control:
Model-Predictive Control (MPC) optimizes a sequence of future actions over a finite horizon, executes only the first action, then replans at each time step - a form of receding horizon control. (Eysenbach, COS435 Lecture 4, 2025, p. 4-5 (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-4-cem-mpc.pdf))

MPC in model-based RL:
Uses a learned or known model to simulate future trajectories, select actions maximizing expected cumulative reward, and adapt by replanning as new data arrives. (Levine, CS285 Lecture 11, 2018, p. 1-18 (https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-11.pdf))

MPC Algorithm (Vanilla)

Algorithm: Model-Predictive Control

Input: Initial state s_0, forward model $f_\psi(s, a)$, reward function $r(s, a)$, action sequence generator $\mathrm{GenActionSeq}()$, planning horizon H, number of candidates N, episode length T

Output: Trajectory $\{s_0, a_0, s_1, a_1, \ldots, s_T\}$

---

1. **for** $t = 0$ **to** $T - 1$ **do**
2. Observe current state: s_t
3. Compute planning horizon: $H_t \leftarrow \min(H, T - t)$
4. **for** $k = 1$ **to** N **do**
5. Generate candidate action sequence: $\mathbf{a}^{(k)} = \mathrm{GenActionSeq}(H_t)$
6. Initialize predicted state: $\hat{s}_0^{(k)} \leftarrow s_t$
7. Initialize cumulative reward: $J^{(k)} \leftarrow 0$
8. **for** $i = 0$ **to** $H_t - 1$ **do**
9. Accumulate reward: $J^{(k)} \leftarrow J^{(k)} + r(\hat{s}_i^{(k)}, a_i^{(k)})$
10. Predict next state: $\hat{s}_{i+1}^{(k)} \leftarrow f_\psi(\hat{s}_i^{(k)}, a_i^{(k)})$
11. **end for**
12. **end for**
13. Select best candidate: $k^* \leftarrow \arg\max_k J^{(k)}$
14. Execute first action: $a_t \leftarrow a_0^{(k^*)}$
15. Observe next state: $s_{t+1} \leftarrow \mathrm{Env.step}(a_t)$
16. **end for**
17. **return** trajectory $\{s_0, a_0, s_1, a_1, \ldots, s_T\}$

Action Sequence Generators:

- **Random shooting:** $\mathrm{GenActionSeq}(H)$ samples H random actions uniformly from action space
- **Policy-based:** $\mathrm{GenActionSeq}(H)$ uses learned policy π and learned forward model to generate action sequences (informed planning)

Key properties:

- **Receding horizon:** Re-plan at every step, execute only first action
- **Model-based planning:** Use forward model f_ψ to predict future states
- **Open-loop planning, closed-loop execution:** Plan without feedback, but re-plan with new observations
- **Flexible action generation:** Compatible with random sampling or policy-guided exploration

In [ ]:

```
# ================================================================
=============
# MPC Implementation with Random Shooting
# ================================================================
=============

def pendulum_reward(state, action):
    """
    Compute Pendulum-v1 reward function.

    Args:
        state: [cos(theta), sin(theta), theta_dot]
        action: Torque value

    Returns:
        reward: -(theta^2 + 0.1*theta_dot^2 + 0.001*action^2)
    """
    cos_theta = state[0]
    sin_theta = state[1]
    theta_dot = state[2]
    theta = np.arctan2(sin_theta, cos_theta)

    # Pendulum reward: penalize angle, angular velocity, and acti
on magnitude
    reward = -(theta**2 + 0.1 * theta_dot**2 + 0.001 * action**2)
    return reward


def random_action_sequence_generator(env, H):
    """Generate random action sequence uniformly from action spac
e"""
    action_sequences = []
    for _ in range(H):
        action = env.action_space.sample()
        action_sequences.append(action)
    return np.array(action_sequences)


def mpc_step(env, forward_model, current_state, H, N):
    """
    Perform one MPC step with random shooting.

    Args:
        env: Gymnasium environment
        forward_model: Trained forward model
        current_state: Current state observation
        H: Planning horizon
        N: Number of random action sequences to sample

    Returns:
        best_action: First action of the best sequence
        best_reward: Total reward of the best sequence
    """
    device = next(forward_model.parameters()).device
    best_reward = -np.inf
    best_action = None
```

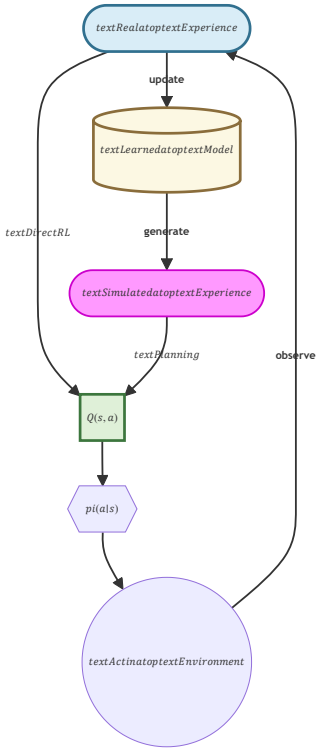
```
    for k in range(N):
        # Generate random action sequence
        action_seq = random_action_sequence_generator(env, H)

        # Rollout using forward model
        state = torch.FloatTensor(current_state).to(device)
        cumulative_reward = 0.0

        for i in range(H):
            action = torch.FloatTensor(action_seq[i]).to(device)

            # Compute reward using Pendulum reward function
            state_np = state.cpu().numpy()
            action_np = action.cpu().numpy()
            reward = pendulum_reward(state_np, action_np)
            cumulative_reward += reward

            # Predict next state using forward model
            with torch.no_grad():
                state_tensor = state.unsqueeze(0)
                action_tensor = action.unsqueeze(0)
                state = forward_model(state_tensor, action_tenso
r).squeeze(0)

        # Track best sequence
        if cumulative_reward > best_reward:
            best_reward = cumulative_reward
            best_action = action_seq[0]

    return best_action, best_reward


def evaluate_mpc_policy(env, forward_model, num_episodes=5, max_s
teps=200, H=10, N=100):
    """
    Evaluate MPC policy on the environment.

    Args:
        env: Gymnasium environment
        forward_model: Trained forward model
        num_episodes: Number of episodes to evaluate
        max_steps: Maximum steps per episode
        H: Planning horizon
        N: Number of random action sequences per MPC step

    Returns:
        episode_rewards: List of total rewards per episode
    """
    episode_rewards = []

    for episode in range(num_episodes):
        state, _ = env.reset()
        total_reward = 0.0

        for t in range(max_steps):
            # Compute effective horizon (don't plan beyond episod
```

```
e end)
            H_t = min(H, max_steps - t)

            # MPC step
            action, predicted_reward = mpc_step(env, forward_mode
l, state, H_t, N)

            # Execute action in real environment
            next_state, reward, terminated, truncated, _ = env.st
ep(action)
            done = terminated or truncated

            total_reward += reward
            state = next_state

            if done:
                break

        episode_rewards.append(total_reward)
        print(f"Episode {episode+1}/{num_episodes}: Total Reward
= {total_reward:.2f}")

    return episode_rewards


# Run MPC evaluation
print("Evaluating MPC with Random Shooting...")

# Create environment with rendering
eval_env = gym.make('Pendulum-v1', render_mode='human')

mpc_rewards = evaluate_mpc_policy(
    env=eval_env,
    forward_model=forward_model,
    num_episodes=5,
    max_steps=200,
    H=20,
    N=20
)

eval_env.close()

print(f"\nMPC Average Reward: {np.mean(mpc_rewards):.2f} ± {np.st
d(mpc_rewards):.2f}")
print(f"MPC Best Episode: {np.max(mpc_rewards):.2f}")
print(f"MPC Worst Episode: {np.min(mpc_rewards):.2f}")
```

```
Evaluating MPC with Random Shooting...
Planning Horizon: H=10, Candidates per step: N=100


c:\Users\micro\miniconda3\envs\rl-lecture\Lib\site-packages\pygam
e\pkgdata.py:25: UserWarning: pkg_resources is deprecated as an A
PI. See https://setuptools.pypa.io/en/latest/pkg_resources.html.
The pkg_resources package is slated for removal as early as 2025-
11-30. Refrain from using this package or pin to Setuptools<81.
  from pkg_resources import resource_stream, resource_exists

Episode 1/5: Total Reward = -1233.34
Episode 2/5: Total Reward = -882.98
Episode 2/5: Total Reward = -882.98
Episode 3/5: Total Reward = -1550.93
Episode 3/5: Total Reward = -1550.93
```

Dyna-Q

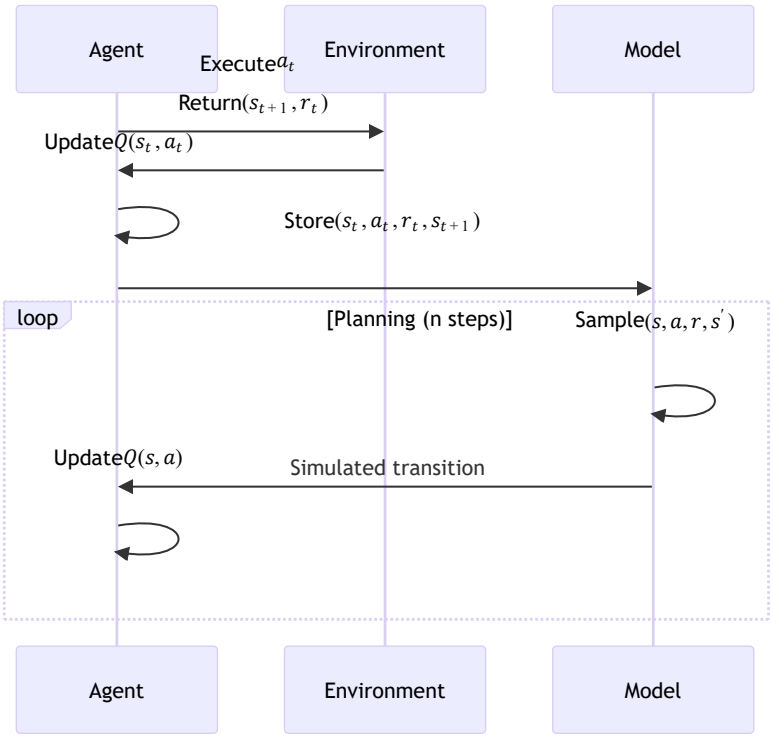
Overview of the Dyna architecture



Integration of planning and learning:
Dyna architecture unifies model-based and model-free RL by learning from both real and simulated experiences, using learned models to generate synthetic transitions for planning while updating value functions from real interactions. (Sutton & Barto, 2018, p. 181-188 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf))

Parallel processes in Dyna:
Planning, acting, model-learning, and direct RL occur in parallel, with the same algorithm applied to both real and simulated experiences, enabling efficient policy improvement. (Sutton & Barto, 2018, p. 181-188 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf))

The Dyna-Q algorithm



Tabular Dyna-Q:
Alternates between real experience (update value function and model) and simulated experience (model generates synthetic transitions), greatly accelerating learning. (Sutton & Barto, 2018, p. 181-188 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf))

Sample model and Q-learning:
Uses sample model for one-step transitions, applies Q-learning update to both real and simulated experiences, converging to optimal policy. (Sutton & Barto, 2018, p. 181-188 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf))

Dyna-Q Algorithm Pseudocode (Tabular Version)

Algorithm: Tabular Dyna-Q

Input:

- α (learning rate for Q-learning)
- γ (discount factor)
- ϵ (exploration parameter for ϵ-greedy)
- n (number of planning steps per real step)

Output: Optimal policy π^* and action-value function $Q(s, a)$

Initialize:

- $Q(s, a) \leftarrow 0$ for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$
- $\text{Model}(s, a) \leftarrow \emptyset$ for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Loop forever:

1. $s \leftarrow$ current (non-terminal) state
2. $a \leftarrow \epsilon$-greedy (s, Q)
3. Take action a; observe resultant reward r and state s'
4. $Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a)\right]$ **(Direct RL)**
5. $\text{Model}(s, a) \leftarrow (r, s')$ **(Model Learning)**
6. **Loop repeat n times: (Planning)**
    - $s \leftarrow$ random previously observed state
    - $a \leftarrow$ random action previously taken in s
    - $r, s' \leftarrow \text{Model}(s, a)$
    - $Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a)\right]$

Key components:

- **Step 4 (Direct RL):** Q-learning update from real experience
- **Step 5 (Model Learning):** Store transition in deterministic model
- **Step 6 (Planning):** n simulated experiences using learned model
- **Same update rule:** Both real (step 4) and simulated (step 6) experiences use identical Q-learning update

(Sutton & Barto, 2018, p. 164 (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf))

Neural Network Dyna-Q Algorithm Pseudocode

Algorithm: Neural Network Dyna-Q

Input:

- α_Q (learning rate for Q-network)
- α_f (learning rate for forward model)
- γ (discount factor)
- ϵ (exploration parameter for ϵ-greedy)
- n (number of planning steps per real step)
- Replay buffer $\mathcal{D}$

Output: Optimal policy π^* via Q-network $Q_\phi(s, a)$ and forward model $f_\psi(s, a)$

Initialize:

- Q-network $Q_\phi(s, a)$ with random parameters ϕ
- Forward model $f_\psi(s, a) \to (r, s')$ with random parameters ψ
- Replay buffer $\mathcal{D} \leftarrow \emptyset$

Loop forever:

1. $s \leftarrow$ current (non-terminal) state
2. $a \leftarrow \epsilon$ -greedy (s, Q_ϕ)
3. Take action a; observe resultant reward r and state s'
4. Store real transition in replay buffer: $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s, a, r, s')\}$
5. Sample minibatch $\mathcal{B}$ from $\mathcal{D}$
6. For each $(s_i, a_i, r_i, s_i') \in \mathcal{B}$:
   - Compute Q-learning target: $y_i \leftarrow r_i + \gamma \max_{a'} Q_\phi(s_i', a')$
   - Update Q-network: $\phi \leftarrow \phi - \alpha_Q \nabla_\phi \big(Q_\phi(s_i, a_i) - y_i\big)^2$ **(Direct RL)**
   - Update forward model: $\psi \leftarrow \psi - \alpha_f \nabla_\psi \big\| f_\psi(s_i, a_i) - (r_i, s_i') \big\|^2$ **(Model Learning)**)
7. **Loop repeat n times: (Planning)**
   - Sample planning minibatch $\mathcal{B}_p$ from $\mathcal{D}$
   - For each $(s_i, a_i, _, _) \in \mathcal{B}_p$:
     - Generate simulated transition: $\hat{r}_i, \hat{s}_i' \leftarrow f_\psi(s_i, a_i)$
     - Compute target: $y_i \leftarrow \hat{r}_i + \gamma \max_{a'} Q_\phi(\hat{s}_i', a')$
     - Update Q-network: $\phi \leftarrow \phi - \alpha_Q \nabla_\phi \big(Q_\phi(s_i, a_i) - y_i\big)^2$

Key differences from tabular version:

- **Q-function:** Table $Q(s, a) \to$ Neural network $Q_\phi(s, a)$

- **Model:** Table $\mathrm{Model}(s, a)$ → Neural network $f_\psi(s, a) \to (r, s')$
- **Updates:** Direct table updates → Gradient descent on loss functions
- **Experience replay:** Added replay buffer $\mathcal{D}$ for stability
- **Batch updates:** Minibatch sampling for all updates (Q-network, forward model, planning)

Key similarities to tabular version:

- **Same three components:** Direct RL & Model Learning (steps 5-6), Planning (step 7)
- **Same Q-learning targets:** $r + \gamma \max_{a'} Q(\cdot)$ used for both real and simulated experience
- **Same planning strategy:** Sample previously observed (s, a) pairs, use model to generate transitions
- **Same integration:** Model learning and Q-learning happen simultaneously from real experience

Neural Network Dyna-Q Implementation

In [ ]:
```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import gymnasium as gym
import matplotlib.pyplot as plt
from collections import deque
import random


# ================================================================
=============
# Q-Network for Discrete Actions
# ================================================================
=============

class DiscreteQNetwork(nn.Module):
    """Q-network for discrete actions: Q_φ(s) → [Q(s,a_0), Q(s,a_
1), ...]"""

    def __init__(self, state_dim, num_actions, hidden_dim=128):
        super(DiscreteQNetwork, self).__init__()
        self.network = nn.Sequential(
            nn.Linear(state_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, num_actions)
        )

    def forward(self, state):
        """Return Q-values for all actions: [Q(s,a_0), Q(s,a_1),
...]"""
        return self.network(state)


# ================================================================
=============
# Dynamics Model for Discrete Actions
# ================================================================
=============

class DiscreteDynamicsModel(nn.Module):
    """Forward model for discrete actions: f_ψ(s, a) → (r, s')

    Takes state and one-hot encoded action as input.
    """

    def __init__(self, state_dim, num_actions, hidden_dim=128):
        super(DiscreteDynamicsModel, self).__init__()
        self.num_actions = num_actions
        # Shared feature extraction
```

```
        self.shared = nn.Sequential(
            nn.Linear(state_dim + num_actions, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU()
        )
        # Separate heads for next state and reward prediction
        self.state_head = nn.Linear(hidden_dim, state_dim)
        self.reward_head = nn.Linear(hidden_dim, 1)

    def forward(self, state, action):
        """Predict reward and next state: (r, s') = f_ψ(s, a)

        Args:
            state: Batch of states (batch_size, state_dim)
            action: Batch of action indices (batch_size,) or one-
hot (batch_size, num_actions)
        """
        # Convert action indices to one-hot if needed
        if action.dim() == 1:
            action_onehot = torch.zeros(action.size(0), self.num_
actions, device=state.device)
            action_onehot.scatter_(1, action.unsqueeze(1).long(),
1)
        else:
            action_onehot = action

        x = torch.cat([state, action_onehot], dim=-1)
        features = self.shared(x)
        predicted_reward = self.reward_head(features)
        predicted_next_state = self.state_head(features)
        return predicted_reward, predicted_next_state


# ================================================================
=============
# Replay Buffer for Discrete Actions
# ================================================================
=============

class DiscreteReplayBuffer:
    """Experience replay buffer for discrete action spaces"""

    def __init__(self, capacity):
        self.buffer = deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, done):
        """Store transition (action is an integer)"""
        self.buffer.append((state, action, reward, next_state, do
ne))

    def sample(self, batch_size):
        """Sample random minibatch"""
        batch = random.sample(self.buffer, batch_size)
        states, actions, rewards, next_states, dones = zip(*batc
h)
        return (
```

```
            np.array(states),
            np.array(actions),
            np.array(rewards),
            np.array(next_states),
            np.array(dones)
        )

    def sample_state_action_pairs(self, batch_size):
        """Sample random state-action pairs for planning"""
        batch = random.sample(self.buffer, batch_size)
        states, actions = zip(*[(s, a) for s, a, _, _, _ in batc
h])
        return np.array(states), np.array(actions)

    def __len__(self):
        return len(self.buffer)


# ==============================================================
=============
# Neural Network Dyna-Q Algorithm
# ==============================================================
=============

def neural_dyna_q(env, num_episodes=100, max_steps=200,
                  alpha_q=1e-3, alpha_f=1e-3, gamma=0.99, epsilon
=0.1,
                  n_planning_steps=5, batch_size=64, planning_bat
ch_size=32,
                  buffer_capacity=10000, hidden_dim=128):
    """
    Neural Network Dyna-Q algorithm for discrete action spaces.

    Args:
        env: Gymnasium environment (discrete action space)
        num_episodes: Number of training episodes
        max_steps: Maximum steps per episode
        alpha_q: Learning rate for Q-network
        alpha_f: Learning rate for forward model
        gamma: Discount factor
        epsilon: Exploration rate for ε-greedy
        n_planning_steps: Number of planning minibatch updates pe
r real step
        batch_size: Minibatch size for Q-network and forward mode
l updates
        planning_batch_size: Minibatch size for planning updates
        buffer_capacity: Replay buffer capacity
        hidden_dim: Hidden layer size for networks

    Returns:
        q_network: Trained Q-network
        dynamics_model: Trained dynamics model
        episode_rewards: List of episode rewards
    """
    # Get environment dimensions
    state_dim = env.observation_space.shape[0]
    num_actions = env.action_space.n
```

```
    # Initialize networks
    q_network = DiscreteQNetwork(state_dim, num_actions, hidden_d
im)
    dynamics_model = DiscreteDynamicsModel(state_dim, num_action
s, hidden_dim)

    # Initialize optimizers
    q_optimizer = optim.Adam(q_network.parameters(), lr=alpha_q)
    f_optimizer = optim.Adam(dynamics_model.parameters(), lr=alph
a_f)

    # Initialize replay buffer
    replay_buffer = DiscreteReplayBuffer(buffer_capacity)

    # Track rewards
    episode_rewards = []

    print("Starting Neural Network Dyna-Q training...")
    print(f"Environment: {env.spec.id}")
    print(f"State dim: {state_dim}, Num actions: {num_actions}")
    print(f"Episodes: {num_episodes}, Planning steps: {n_planning
_steps}")
    print(f"Batch size: {batch_size}, Planning batch size: {plann
ing_batch_size}")
    print()

    # Training loop
    for episode in range(num_episodes):
        state, _ = env.reset()
        episode_reward = 0

        for t in range(max_steps):
            # Step 2: ε-greedy action selection
            if np.random.rand() < epsilon:
                action = env.action_space.sample()
            else:
                with torch.no_grad():
                    state_tensor = torch.FloatTensor(state).unsqu
eeze(0)
                    q_values = q_network(state_tensor)
                    action = q_values.argmax(dim=1).item()

            # Step 3: Execute action and observe
            next_state, reward, terminated, truncated, _ = env.st
ep(action)
            done = terminated or truncated
            episode_reward += reward

            # Step 4: Store transition in replay buffer
            replay_buffer.push(state, action, reward, next_state,
done)

            # Step 5-6: Direct RL & Model Learning - Update both
networks from same minibatch
            if len(replay_buffer) >= batch_size:
                states, actions, rewards, next_states, dones = re
```

```
play_buffer.sample(batch_size)

                # Convert to tensors
                states_tensor = torch.FloatTensor(states)
                actions_tensor = torch.LongTensor(actions)
                rewards_tensor = torch.FloatTensor(rewards)
                next_states_tensor = torch.FloatTensor(next_state
s)
                dones_tensor = torch.FloatTensor(dones)

                # ================================================
=============
                # Direct RL: Update Q-network
                # ================================================
=============

                # Compute Q-values for current state-action pairs
                q_values_all = q_network(states_tensor)
                q_values = q_values_all.gather(1, actions_tensor.
unsqueeze(1)).squeeze(1)

                # Compute targets: y = r + γ max_a' Q(s', a')
                with torch.no_grad():
                    next_q_values_all = q_network(next_states_ten
sor)
                    next_q_values = next_q_values_all.max(dim=1)
[0]
                    targets = rewards_tensor + gamma * next_q_val
ues * (1 - dones_tensor)

                # Update Q-network
                q_loss = nn.MSELoss()(q_values, targets)
                q_optimizer.zero_grad()
                q_loss.backward()
                q_optimizer.step()

                # ================================================
=============
                # Model Learning: Update dynamics model
                # ================================================
=============

                # Predict reward and next state for entire miniba
tch
                pred_rewards, pred_next_states = dynamics_model(s
tates_tensor, actions_tensor)

                # Compute loss: MSE for both reward and next stat
e
                reward_loss = nn.MSELoss()(pred_rewards.squeeze
(), rewards_tensor)
                state_loss = nn.MSELoss()(pred_next_states, next_
states_tensor)
                model_loss = reward_loss + state_loss

                # Update dynamics model
                f_optimizer.zero_grad()
```

```
                model_loss.backward()
                f_optimizer.step()

            # Step 7: Planning - Update Q-network from simulated
experience using minibatches
            if len(replay_buffer) >= planning_batch_size:
                for _ in range(n_planning_steps):
                    # Sample planning minibatch from buffer
                    planning_states, planning_actions = replay_bu
ffer.sample_state_action_pairs(planning_batch_size)

                    planning_states_tensor = torch.FloatTensor(pl
anning_states)
                    planning_actions_tensor = torch.LongTensor(pl
anning_actions)

                    # Generate simulated transitions using dynami
cs model
                    with torch.no_grad():
                        sim_rewards, sim_next_states = dynamics_m
odel(planning_states_tensor, planning_actions_tensor)

                    # Compute Q-values for planning state-action
pairs
                    q_values_all = q_network(planning_states_tens
or)
                    q_values = q_values_all.gather(1, planning_ac
tions_tensor.unsqueeze(1)).squeeze(1)

                    # Compute targets: y = r + γ max_a' Q(s', a')
                    with torch.no_grad():
                        sim_next_q_values_all = q_network(sim_nex
t_states)
                        sim_next_q_values = sim_next_q_values_al
l.max(dim=1)[0]
                        planning_targets = sim_rewards.squeeze()
+ gamma * sim_next_q_values

                    # Update Q-network
                    planning_loss = nn.MSELoss()(q_values, planni
ng_targets)
                    q_optimizer.zero_grad()
                    planning_loss.backward()
                    q_optimizer.step()

            # Move to next state
            state = next_state

            if done:
                break

        # Track episode reward
        episode_rewards.append(episode_reward)

        # Print progress
        if (episode + 1) % 10 == 0:
            avg_reward = np.mean(episode_rewards[-10:])
```

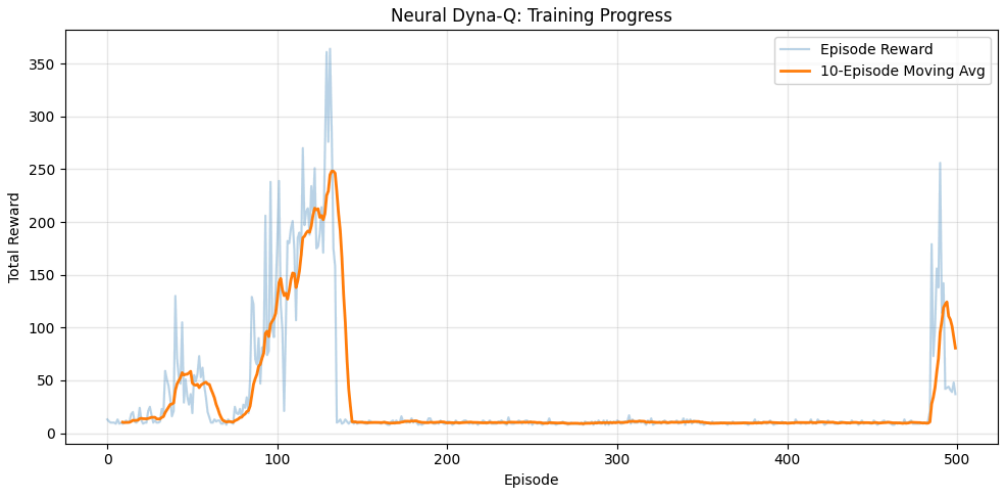
```
            print(f"Episode {episode + 1}/{num_episodes}, Avg Rew
ard (last 10): {avg_reward:.2f}")

    print("\nNeural Network Dyna-Q training complete!")
    return q_network, dynamics_model, episode_rewards
```

In [10]:

```
# Train Neural Network Dyna-Q on CartPole
print("=" * 60)
print("Neural Network Dyna-Q on CartPole-v1")
print("=" * 60)

# Create environment
env_dyna = gym.make('CartPole-v1')

# Train Neural Dyna-Q
q_network, dynamics_model, dyna_rewards = neural_dyna_q(
    env=env_dyna,
    num_episodes=500,
    max_steps=500,
    alpha_q=1e-3,
    alpha_f=1e-3,
    gamma=0.99,
    epsilon=0.1,
    n_planning_steps=1,
    batch_size=4,
    planning_batch_size=16,
    buffer_capacity=10000,
    hidden_dim=64
)

# Reload helper functions and import plotting function
import importlib
import helper_functions
importlib.reload(helper_functions)
from helper_functions import plot_dyna_q_training

# Plot training progress
plot_dyna_q_training(dyna_rewards, window_size=10)
plt.show()

print(f"\nFinal Statistics:")
print(f"  Average reward (last 10 episodes): {np.mean(dyna_reward
s[-10:]):.2f}")
print(f"  Average reward (last 20 episodes): {np.mean(dyna_reward
s[-20:]):.2f}")
print(f"  Best episode reward: {np.max(dyna_rewards):.2f}")

env_dyna.close()
```

```
============================================================
Neural Network Dyna-Q on CartPole-v1
============================================================
Starting Neural Network Dyna-Q training...
Environment: CartPole-v1
State dim: 4, Num actions: 2
Episodes: 500, Planning steps: 1
Batch size: 4, Planning batch size: 16

Episode 10/500, Avg Reward (last 10): 10.40
Episode 10/500, Avg Reward (last 10): 10.40
Episode 20/500, Avg Reward (last 10): 13.80
Episode 20/500, Avg Reward (last 10): 13.80
Episode 30/500, Avg Reward (last 10): 13.60
Episode 30/500, Avg Reward (last 10): 13.60
Episode 40/500, Avg Reward (last 10): 28.70
Episode 40/500, Avg Reward (last 10): 28.70
Episode 50/500, Avg Reward (last 10): 58.60
Episode 50/500, Avg Reward (last 10): 58.60
Episode 60/500, Avg Reward (last 10): 46.60
Episode 60/500, Avg Reward (last 10): 46.60
Episode 70/500, Avg Reward (last 10): 11.30
Episode 70/500, Avg Reward (last 10): 11.30
Episode 80/500, Avg Reward (last 10): 15.10
Episode 80/500, Avg Reward (last 10): 15.10
Episode 90/500, Avg Reward (last 10): 63.30
Episode 90/500, Avg Reward (last 10): 63.30
Episode 100/500, Avg Reward (last 10): 113.40
Episode 100/500, Avg Reward (last 10): 113.40
Episode 110/500, Avg Reward (last 10): 151.70
Episode 110/500, Avg Reward (last 10): 151.70
Episode 120/500, Avg Reward (last 10): 190.20
Episode 120/500, Avg Reward (last 10): 190.20
Episode 130/500, Avg Reward (last 10): 225.10
Episode 130/500, Avg Reward (last 10): 225.10
Episode 140/500, Avg Reward (last 10): 131.10
Episode 140/500, Avg Reward (last 10): 131.10
Episode 150/500, Avg Reward (last 10): 10.40
Episode 150/500, Avg Reward (last 10): 10.40
Episode 160/500, Avg Reward (last 10): 10.20
Episode 160/500, Avg Reward (last 10): 10.20
Episode 170/500, Avg Reward (last 10): 9.70
Episode 170/500, Avg Reward (last 10): 9.70
Episode 180/500, Avg Reward (last 10): 11.60
Episode 180/500, Avg Reward (last 10): 11.60
Episode 190/500, Avg Reward (last 10): 10.00
Episode 190/500, Avg Reward (last 10): 10.00
Episode 200/500, Avg Reward (last 10): 10.10
Episode 200/500, Avg Reward (last 10): 10.10
Episode 210/500, Avg Reward (last 10): 10.10
Episode 210/500, Avg Reward (last 10): 10.10
Episode 220/500, Avg Reward (last 10): 10.00
Episode 220/500, Avg Reward (last 10): 10.00
Episode 230/500, Avg Reward (last 10): 10.00
Episode 230/500, Avg Reward (last 10): 10.00
Episode 240/500, Avg Reward (last 10): 10.40
```

```
Episode 240/500, Avg Reward (last 10): 10.40
Episode 250/500, Avg Reward (last 10): 10.60
Episode 250/500, Avg Reward (last 10): 10.60
Episode 260/500, Avg Reward (last 10): 9.40
Episode 260/500, Avg Reward (last 10): 9.40
Episode 270/500, Avg Reward (last 10): 9.70
Episode 270/500, Avg Reward (last 10): 9.70
Episode 280/500, Avg Reward (last 10): 9.20
Episode 280/500, Avg Reward (last 10): 9.20
Episode 290/500, Avg Reward (last 10): 9.90
Episode 290/500, Avg Reward (last 10): 9.90
Episode 300/500, Avg Reward (last 10): 9.90
Episode 300/500, Avg Reward (last 10): 9.90
Episode 310/500, Avg Reward (last 10): 11.30
Episode 310/500, Avg Reward (last 10): 11.30
Episode 320/500, Avg Reward (last 10): 10.30
Episode 320/500, Avg Reward (last 10): 10.30
Episode 330/500, Avg Reward (last 10): 10.30
Episode 330/500, Avg Reward (last 10): 10.30
Episode 340/500, Avg Reward (last 10): 10.80
Episode 340/500, Avg Reward (last 10): 10.80
Episode 350/500, Avg Reward (last 10): 10.50
Episode 350/500, Avg Reward (last 10): 10.50
Episode 360/500, Avg Reward (last 10): 9.40
Episode 360/500, Avg Reward (last 10): 9.40
Episode 370/500, Avg Reward (last 10): 9.50
Episode 370/500, Avg Reward (last 10): 9.50
Episode 380/500, Avg Reward (last 10): 9.50
Episode 380/500, Avg Reward (last 10): 9.50
Episode 390/500, Avg Reward (last 10): 10.20
Episode 390/500, Avg Reward (last 10): 10.20
Episode 400/500, Avg Reward (last 10): 9.80
Episode 400/500, Avg Reward (last 10): 9.80
Episode 410/500, Avg Reward (last 10): 10.00
Episode 410/500, Avg Reward (last 10): 10.00
Episode 420/500, Avg Reward (last 10): 10.40
Episode 420/500, Avg Reward (last 10): 10.40
Episode 430/500, Avg Reward (last 10): 9.80
Episode 430/500, Avg Reward (last 10): 9.80
Episode 440/500, Avg Reward (last 10): 10.10
Episode 440/500, Avg Reward (last 10): 10.10
Episode 450/500, Avg Reward (last 10): 9.80
Episode 450/500, Avg Reward (last 10): 9.80
Episode 460/500, Avg Reward (last 10): 10.20
Episode 460/500, Avg Reward (last 10): 10.20
Episode 470/500, Avg Reward (last 10): 9.60
Episode 470/500, Avg Reward (last 10): 9.60
Episode 480/500, Avg Reward (last 10): 9.70
Episode 480/500, Avg Reward (last 10): 9.70
Episode 490/500, Avg Reward (last 10): 70.70
Episode 490/500, Avg Reward (last 10): 70.70
Episode 500/500, Avg Reward (last 10): 80.60

Neural Network Dyna-Q training complete!
Episode 500/500, Avg Reward (last 10): 80.60

Neural Network Dyna-Q training complete!
```




```
Final Statistics:
  Average reward (last 10 episodes): 80.60
  Average reward (last 20 episodes): 75.65
  Best episode reward: 364.00
```

Comparing Planning Steps: Impact on Learning Efficiency

To understand the impact of planning on sample efficiency, let's compare Neural Dyna-Q with different amounts of planning:

- **0 planning steps:** Pure model-free Q-learning (only learns from real experience)
- **1 planning step:** Modest model-based augmentation
- **2 planning steps:** More aggressive planning

This comparison will reveal how planning accelerates learning while highlighting the risk of over-planning with inaccurate models.

In [ ]:

```
# Comparison: Neural Dyna-Q with Different Planning Steps
print("=" * 70)
print("COMPARISON: Impact of Planning Steps on Learning Efficienc
y")
print("=" * 70)
print()

# Store results for comparison
planning_results = {}
planning_configs = [
    (0, "0 planning steps (Pure Q-learning)"),
    (1, "1 planning step"),
    (2, "2 planning steps")
]

# Common hyperparameters
common_params = {
    'num_episodes': 200,
    'max_steps': 500,
    'alpha_q': 1e-3,
    'alpha_f': 1e-3,
    'gamma': 0.99,
    'epsilon': 0.1,
    'batch_size': 4,
    'planning_batch_size': 4,
    'buffer_capacity': 100000,
    'hidden_dim': 64
}

# Train with different planning steps
for n_planning, description in planning_configs:
    print(f"\n{'='*70}")
    print(f"Training: {description}")
    print(f"{'='*70}")

    # Create fresh environment for each run
    env_compare = gym.make('CartPole-v1')

    # Train Neural Dyna-Q
    q_net, dyn_model, rewards = neural_dyna_q(
        env=env_compare,
        n_planning_steps=n_planning,
        **common_params
    )

    # Store results
    planning_results[description] = rewards

    # Print summary statistics
    print(f"\nResults for {description}:")
    print(f"  Final 10 episodes avg: {np.mean(rewards[-10:]):.2
f}")
    print(f"  Final 20 episodes avg: {np.mean(rewards[-20:]):.2
f}")
    print(f"  Best episode: {np.max(rewards):.2f}")
    print(f"  Overall average: {np.mean(rewards):.2f}")
```

```
    env_compare.close()

# Plot comparison using the new multi-plot function
print(f"\n{'='*70}")
print("Generating Comparison Plot")
print(f"{'='*70}")

# Reload helper functions to get the new plotting function
import importlib
import helper_functions
importlib.reload(helper_functions)
from helper_functions import plot_dyna_q_multiple

# Create comparison plot
fig, ax = plot_dyna_q_multiple(
    planning_results,
    window_size=10,
    title='Neural Dyna-Q: Impact of Planning Steps on Learning'
)
plt.show()

# Print comparative analysis
print("\n" + "="*70)
print("COMPARATIVE ANALYSIS")
print("="*70)

for description, rewards in planning_results.items():
    episodes_to_threshold = None
    threshold = 200  # Consider this "good performance" for CartP
ole

    for i, r in enumerate(rewards):
        if r >= threshold:
            episodes_to_threshold = i + 1
            break

    print(f"\n{description}:")
    print(f"  Episodes to reach {threshold} reward: {episodes_to_
threshold if episodes_to_threshold else 'Not reached'}")
    print(f"  Average reward (first 50 episodes): {np.mean(reward
s[:50]):.2f}")
    print(f"  Average reward (last 50 episodes): {np.mean(rewards
[-50:]):.2f}")
    print(f"  Standard deviation (last 50): {np.std(rewards[-5
0:]):.2f}")
```

```
=================================================================
=====
COMPARISON: Impact of Planning Steps on Learning Efficiency
=================================================================
=====


=================================================================
=====
Training: 0 planning steps (Pure Q-learning)
=================================================================
=====
Starting Neural Network Dyna-Q training...
Environment: CartPole-v1
State dim: 4, Num actions: 2
Episodes: 300, Planning steps: 0
Batch size: 4, Planning batch size: 16

Episode 10/300, Avg Reward (last 10): 10.00
Episode 20/300, Avg Reward (last 10): 10.10
Episode 20/300, Avg Reward (last 10): 10.10
Episode 30/300, Avg Reward (last 10): 11.60
Episode 30/300, Avg Reward (last 10): 11.60
Episode 40/300, Avg Reward (last 10): 16.10
Episode 40/300, Avg Reward (last 10): 16.10
Episode 50/300, Avg Reward (last 10): 14.50
Episode 60/300, Avg Reward (last 10): 12.40
Episode 50/300, Avg Reward (last 10): 14.50
Episode 60/300, Avg Reward (last 10): 12.40
Episode 70/300, Avg Reward (last 10): 22.30
Episode 70/300, Avg Reward (last 10): 22.30
Episode 80/300, Avg Reward (last 10): 20.20
Episode 80/300, Avg Reward (last 10): 20.20
Episode 90/300, Avg Reward (last 10): 49.50
Episode 90/300, Avg Reward (last 10): 49.50
Episode 100/300, Avg Reward (last 10): 24.90
Episode 100/300, Avg Reward (last 10): 24.90
Episode 110/300, Avg Reward (last 10): 41.60
Episode 110/300, Avg Reward (last 10): 41.60
Episode 120/300, Avg Reward (last 10): 36.90
Episode 120/300, Avg Reward (last 10): 36.90
Episode 130/300, Avg Reward (last 10): 82.90
Episode 130/300, Avg Reward (last 10): 82.90
Episode 140/300, Avg Reward (last 10): 74.80
Episode 140/300, Avg Reward (last 10): 74.80
Episode 150/300, Avg Reward (last 10): 131.90
Episode 150/300, Avg Reward (last 10): 131.90
Episode 160/300, Avg Reward (last 10): 157.00
Episode 160/300, Avg Reward (last 10): 157.00
Episode 170/300, Avg Reward (last 10): 249.60
Episode 170/300, Avg Reward (last 10): 249.60
Episode 180/300, Avg Reward (last 10): 278.70
Episode 180/300, Avg Reward (last 10): 278.70
Episode 190/300, Avg Reward (last 10): 275.10
Episode 190/300, Avg Reward (last 10): 275.10
Episode 200/300, Avg Reward (last 10): 268.40
```

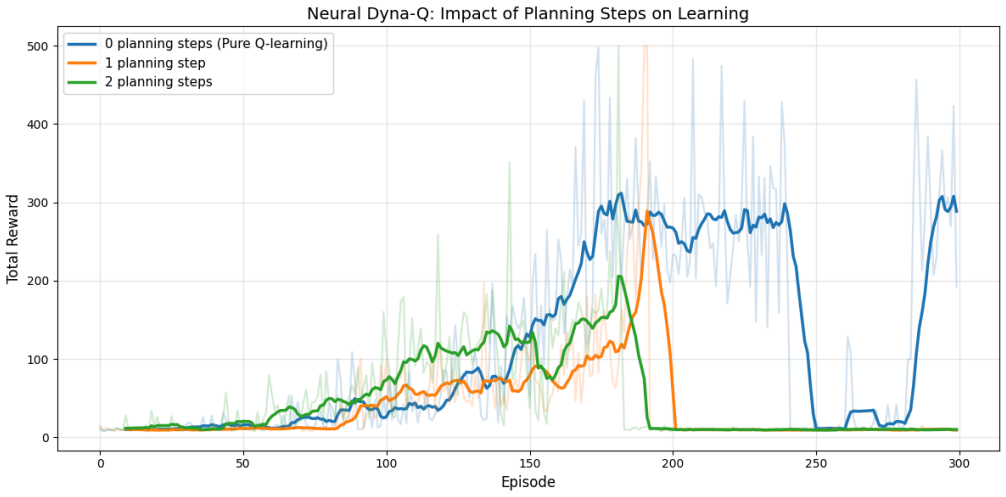
```
Episode 200/300, Avg Reward (last 10): 268.40
Episode 210/300, Avg Reward (last 10): 265.30
Episode 210/300, Avg Reward (last 10): 265.30
Episode 220/300, Avg Reward (last 10): 275.40
Episode 220/300, Avg Reward (last 10): 275.40
Episode 230/300, Avg Reward (last 10): 268.40
Episode 230/300, Avg Reward (last 10): 268.40
Episode 240/300, Avg Reward (last 10): 298.10
Episode 240/300, Avg Reward (last 10): 298.10
Episode 250/300, Avg Reward (last 10): 29.80
Episode 260/300, Avg Reward (last 10): 11.30
Episode 250/300, Avg Reward (last 10): 29.80
Episode 260/300, Avg Reward (last 10): 11.30
Episode 270/300, Avg Reward (last 10): 34.10
Episode 270/300, Avg Reward (last 10): 34.10
Episode 280/300, Avg Reward (last 10): 20.20
Episode 280/300, Avg Reward (last 10): 20.20
Episode 290/300, Avg Reward (last 10): 221.80
Episode 290/300, Avg Reward (last 10): 221.80
Episode 300/300, Avg Reward (last 10): 288.50

Neural Network Dyna-Q training complete!

Results for 0 planning steps (Pure Q-learning):
  Final 10 episodes avg: 288.50
  Final 20 episodes avg: 255.15
  Best episode: 498.00
  Overall average: 116.71

================================================================
=====
Training: 1 planning step
================================================================
=====
Starting Neural Network Dyna-Q training...
Environment: CartPole-v1
State dim: 4, Num actions: 2
Episodes: 300, Planning steps: 1
Batch size: 4, Planning batch size: 16

Episode 300/300, Avg Reward (last 10): 288.50

Neural Network Dyna-Q training complete!

Results for 0 planning steps (Pure Q-learning):
  Final 10 episodes avg: 288.50
  Final 20 episodes avg: 255.15
  Best episode: 498.00
  Overall average: 116.71

================================================================
=====
Training: 1 planning step
================================================================
=====
Starting Neural Network Dyna-Q training...
Environment: CartPole-v1
```

```
State dim: 4, Num actions: 2
Episodes: 300, Planning steps: 1
Batch size: 4, Planning batch size: 16

Episode 10/300, Avg Reward (last 10): 10.50
Episode 10/300, Avg Reward (last 10): 10.50
Episode 20/300, Avg Reward (last 10): 9.40
Episode 20/300, Avg Reward (last 10): 9.40
Episode 30/300, Avg Reward (last 10): 9.90
Episode 30/300, Avg Reward (last 10): 9.90
Episode 40/300, Avg Reward (last 10): 9.80
Episode 40/300, Avg Reward (last 10): 9.80
Episode 50/300, Avg Reward (last 10): 11.50
Episode 50/300, Avg Reward (last 10): 11.50
Episode 60/300, Avg Reward (last 10): 11.00
Episode 60/300, Avg Reward (last 10): 11.00
Episode 70/300, Avg Reward (last 10): 12.40
Episode 70/300, Avg Reward (last 10): 12.40
Episode 80/300, Avg Reward (last 10): 10.80
Episode 80/300, Avg Reward (last 10): 10.80
Episode 90/300, Avg Reward (last 10): 25.00
Episode 90/300, Avg Reward (last 10): 25.00
Episode 100/300, Avg Reward (last 10): 49.30
Episode 100/300, Avg Reward (last 10): 49.30
Episode 110/300, Avg Reward (last 10): 61.20
Episode 110/300, Avg Reward (last 10): 61.20
Episode 120/300, Avg Reward (last 10): 65.40
Episode 120/300, Avg Reward (last 10): 65.40
Episode 130/300, Avg Reward (last 10): 59.20
Episode 130/300, Avg Reward (last 10): 59.20
Episode 140/300, Avg Reward (last 10): 75.20
Episode 140/300, Avg Reward (last 10): 75.20
Episode 150/300, Avg Reward (last 10): 73.30
Episode 150/300, Avg Reward (last 10): 73.30
Episode 160/300, Avg Reward (last 10): 68.30
Episode 160/300, Avg Reward (last 10): 68.30
Episode 170/300, Avg Reward (last 10): 91.90
Episode 170/300, Avg Reward (last 10): 91.90
Episode 180/300, Avg Reward (last 10): 120.80
Episode 180/300, Avg Reward (last 10): 120.80
Episode 190/300, Avg Reward (last 10): 204.20
Episode 190/300, Avg Reward (last 10): 204.20
Episode 200/300, Avg Reward (last 10): 107.90
Episode 200/300, Avg Reward (last 10): 107.90
Episode 210/300, Avg Reward (last 10): 9.90
Episode 210/300, Avg Reward (last 10): 9.90
Episode 220/300, Avg Reward (last 10): 9.70
Episode 220/300, Avg Reward (last 10): 9.70
Episode 230/300, Avg Reward (last 10): 9.70
Episode 230/300, Avg Reward (last 10): 9.70
Episode 240/300, Avg Reward (last 10): 9.30
Episode 240/300, Avg Reward (last 10): 9.30
Episode 250/300, Avg Reward (last 10): 9.60
Episode 250/300, Avg Reward (last 10): 9.60
Episode 260/300, Avg Reward (last 10): 9.50
Episode 260/300, Avg Reward (last 10): 9.50
Episode 270/300, Avg Reward (last 10): 10.10
```

```
Episode 270/300, Avg Reward (last 10): 10.10
Episode 280/300, Avg Reward (last 10): 9.20
Episode 280/300, Avg Reward (last 10): 9.20
Episode 290/300, Avg Reward (last 10): 10.30
Episode 290/300, Avg Reward (last 10): 10.30
Episode 300/300, Avg Reward (last 10): 9.40

Neural Network Dyna-Q training complete!

Results for 1 planning step:
  Final 10 episodes avg: 9.40
  Final 20 episodes avg: 9.85
  Best episode: 500.00
  Overall average: 39.46

================================================================
=====
Training: 2 planning steps
================================================================
=====
Starting Neural Network Dyna-Q training...
Environment: CartPole-v1
State dim: 4, Num actions: 2
Episodes: 300, Planning steps: 2
Batch size: 4, Planning batch size: 16

Episode 300/300, Avg Reward (last 10): 9.40

Neural Network Dyna-Q training complete!

Results for 1 planning step:
  Final 10 episodes avg: 9.40
  Final 20 episodes avg: 9.85
  Best episode: 500.00
  Overall average: 39.46

================================================================
=====
Training: 2 planning steps
================================================================
=====
Starting Neural Network Dyna-Q training...
Environment: CartPole-v1
State dim: 4, Num actions: 2
Episodes: 300, Planning steps: 2
Batch size: 4, Planning batch size: 16

Episode 10/300, Avg Reward (last 10): 11.60
Episode 10/300, Avg Reward (last 10): 11.60
Episode 20/300, Avg Reward (last 10): 13.50
Episode 20/300, Avg Reward (last 10): 13.50
Episode 30/300, Avg Reward (last 10): 13.10
Episode 30/300, Avg Reward (last 10): 13.10
Episode 40/300, Avg Reward (last 10): 11.00
Episode 40/300, Avg Reward (last 10): 11.00
Episode 50/300, Avg Reward (last 10): 19.90
Episode 50/300, Avg Reward (last 10): 19.90
```

```
Episode 60/300, Avg Reward (last 10): 22.90
Episode 60/300, Avg Reward (last 10): 22.90
Episode 70/300, Avg Reward (last 10): 30.20
Episode 70/300, Avg Reward (last 10): 30.20
Episode 80/300, Avg Reward (last 10): 47.30
Episode 80/300, Avg Reward (last 10): 47.30
Episode 90/300, Avg Reward (last 10): 43.70
Episode 90/300, Avg Reward (last 10): 43.70
Episode 100/300, Avg Reward (last 10): 71.20
Episode 100/300, Avg Reward (last 10): 71.20
Episode 110/300, Avg Reward (last 10): 99.40
Episode 110/300, Avg Reward (last 10): 99.40
Episode 120/300, Avg Reward (last 10): 114.20
Episode 120/300, Avg Reward (last 10): 114.20
Episode 130/300, Avg Reward (last 10): 108.00
Episode 130/300, Avg Reward (last 10): 108.00
Episode 140/300, Avg Reward (last 10): 131.60
Episode 140/300, Avg Reward (last 10): 131.60
Episode 150/300, Avg Reward (last 10): 121.00
Episode 150/300, Avg Reward (last 10): 121.00
Episode 160/300, Avg Reward (last 10): 79.50
Episode 160/300, Avg Reward (last 10): 79.50
Episode 170/300, Avg Reward (last 10): 151.10
Episode 170/300, Avg Reward (last 10): 151.10
Episode 180/300, Avg Reward (last 10): 161.60
Episode 180/300, Avg Reward (last 10): 161.60
Episode 190/300, Avg Reward (last 10): 90.40
Episode 190/300, Avg Reward (last 10): 90.40
Episode 200/300, Avg Reward (last 10): 10.70
Episode 200/300, Avg Reward (last 10): 10.70
Episode 210/300, Avg Reward (last 10): 9.90
Episode 210/300, Avg Reward (last 10): 9.90
Episode 220/300, Avg Reward (last 10): 9.70
Episode 220/300, Avg Reward (last 10): 9.70
Episode 230/300, Avg Reward (last 10): 9.80
Episode 230/300, Avg Reward (last 10): 9.80
Episode 240/300, Avg Reward (last 10): 10.20
Episode 240/300, Avg Reward (last 10): 10.20
Episode 250/300, Avg Reward (last 10): 9.40
Episode 250/300, Avg Reward (last 10): 9.40
Episode 260/300, Avg Reward (last 10): 10.10
Episode 260/300, Avg Reward (last 10): 10.10
Episode 270/300, Avg Reward (last 10): 10.30
Episode 270/300, Avg Reward (last 10): 10.30
Episode 280/300, Avg Reward (last 10): 10.20
Episode 280/300, Avg Reward (last 10): 10.20
Episode 290/300, Avg Reward (last 10): 9.60
Episode 290/300, Avg Reward (last 10): 9.60
Episode 300/300, Avg Reward (last 10): 10.00

Neural Network Dyna-Q training complete!

Results for 2 planning steps:
  Final 10 episodes avg: 10.00
  Final 20 episodes avg: 9.80
  Best episode: 500.00
  Overall average: 48.37
```

```
================================================================
=====
Generating Comparison Plot
================================================================
=====
Episode 300/300, Avg Reward (last 10): 10.00

Neural Network Dyna-Q training complete!

Results for 2 planning steps:
  Final 10 episodes avg: 10.00
  Final 20 episodes avg: 9.80
  Best episode: 500.00
  Overall average: 48.37

================================================================
=====
Generating Comparison Plot
================================================================
=====
```



```
================================================================
=====
COMPARATIVE ANALYSIS
================================================================
=====

0 planning steps (Pure Q-learning):
  Episodes to reach 200 reward: 153
  Average reward (first 50 episodes): 12.46
  Average reward (last 50 episodes): 115.18
  Standard deviation (last 50): 139.04

1 planning step:
  Episodes to reach 200 reward: 185
  Average reward (first 50 episodes): 10.22
  Average reward (last 50 episodes): 9.70
  Standard deviation (last 50): 1.27

2 planning steps:
  Episodes to reach 200 reward: 119
  Average reward (first 50 episodes): 13.82
  Average reward (last 50 episodes): 10.04
  Standard deviation (last 50): 1.34
```

Summary

In this lecture, we explored **Model-Based Reinforcement Learning**, bridging the gap between model-free methods and planning-based approaches by learning forward models of environment dynamics.

Key Concepts Covered:

1. Forward Models in Cognition and RL

- Forward models predict consequences of actions - central to human sensorimotor control and planning
- Neuroscience evidence: Hippocampus simulates future paths, OFC represents reward values
- Model-based RL uses learned models for goal-directed planning vs. model-free reactive behavior

2. Learning Forward Models as Supervised Learning

- Forward model $f_\psi(s, a) \to s'$ approximates MDP transition dynamics $p(s'|s, a)$
- Trained via supervised learning: minimize MSE between predicted and observed next states
- Online learning algorithm: collect transitions with policy, update model after each step
- **Key challenge:** Prediction errors accumulate over multi-step rollouts

3. Techniques for Improving Model Accuracy

- **Delta prediction:** Predict state changes Δs instead of full states (zero-centered, better stability)
- **Ensemble methods:** Average predictions across multiple models (reduces overfitting)
- **Data aggregation:** Retrain on current policy distribution to match state visitation

4. Model-Predictive Control (MPC)

- Receding horizon control: optimize action sequence over finite horizon, execute first action, replan
- **Random shooting:** Generate random action sequences, rollout with model, select best
- **Open-loop planning, closed-loop execution:** Plan without feedback, but adapt via replanning
- **Trade-off:** Long horizons accumulate model errors; short horizons limit foresight

5. Dyna-Q Architecture

- **Three parallel components:**
    1. **Direct RL:** Update Q-function from real experience
    2. **Model Learning:** Learn dynamics model from real transitions
    3. **Planning:** Generate simulated experience using model, update Q-function
- **Key insight:** Same Q-learning update for both real and simulated experience
- **Sample efficiency:** Each real transition enables multiple model-based updates

6. Neural Network Dyna-Q

- Extends tabular Dyna-Q to continuous states using function approximation
- **DiscreteQNetwork:** $Q_\phi(s) \to [Q(s, a_0), Q(s, a_1), \dots]$ for all actions
- **DiscreteDynamicsModel:** $f_\psi(s, a) \to (r, s')$ predicts reward and next state
- **Experience replay:** Stabilizes training by sampling minibatches from buffer
- **Minibatch planning:** Sample (s, a) pairs, generate simulated transitions, update Q-network

What You Can Do Now:

✓ Train forward models as supervised learning problems from interaction data
✓ Implement Model-Predictive Control for planning with learned models
✓ Build Dyna-Q agents that learn from both real and simulated experience
✓ Diagnose model-based learning failures (catastrophic forgetting, error accumulation)
✓ Apply neural networks to both Q-functions and dynamics models

Outlook: Intrinsic Motivation and Entropy-Regularized RL

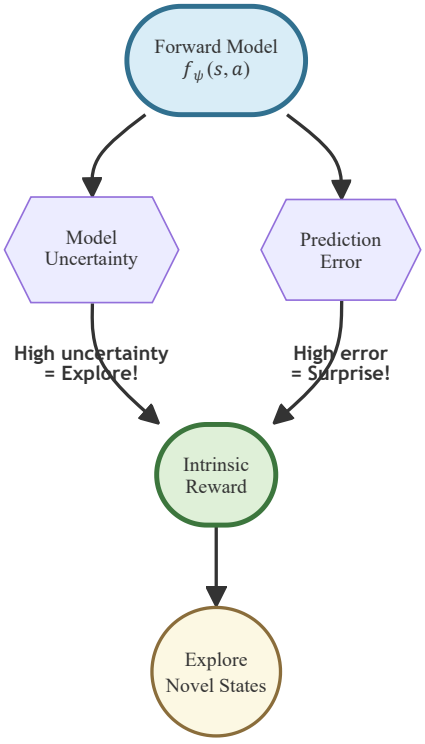
Beyond ϵ-Greedy Exploration: In this and previous lectures, we relied on simple exploration strategies like ϵ-greedy random action selection. While effective for basic tasks, these methods have significant limitations:

1. **Uninformed Exploration:** Random actions don't consider which states are novel or informative
2. **Inefficient Coverage:** May revisit known states repeatedly while missing important regions
3. **No Curiosity:** Doesn't actively seek out surprising or uncertain situations
4. **Fixed Strategy:** Exploration rate ϵ constant, doesn't adapt to learning progress

How Can We Do Better?

Forward Models Enable Smarter Exploration! 🎯

The forward models we learned in this lecture are not just for planning - they're powerful tools for **intelligent exploration**:



L10 Preview: Advanced Exploration Methods

1. Curiosity-Driven Exploration

- **Idea:** Reward agent for visiting states where forward model has high prediction error
- **Intrinsic reward:** $r_{\text{intrinsic}} = \|f_\psi(s, a) - s'\|^2$ (prediction error as curiosity signal)
- **Benefit:** Naturally explores novel states where model is uncertain

- **Application:** Exploration in sparse-reward environments, video game level discovery

2. Entropy-Regularized RL (Maximum Entropy Framework)

- **Idea:** Maximize both reward **and** policy entropy $H(\pi)$
- **Objective:** $\max_\pi \mathbb{E}\left[\sum_t r_t + \alpha H(\pi(\cdot|s_t))\right]$
- **Benefit:** Naturally stochastic policies explore diverse actions
- **Connection:** Soft Q-learning, Soft Actor-Critic (SAC), Proximal Policy Optimization (PPO)
- **Application:** Robustness to disturbances, multi-modal behaviors